CODE SPORT

Sandya Mannarswamy

In this month's column, we continue our discussion on detecting duplicate questions in community question-answering forums.

Let's continue exploring the topic we started out on in last month's column, in which we discussed the problem of detecting duplicate questions in community question answering (CQA) forums using Quora's question pair data set.

Given a pair of questions <Q1, Q2>, the task is to identify whether Q2 is a duplicate of Q1. Our system for duplicate detection first needs to create a representation for each input sentence, and then feed the representations for each of the two questions to a classifier which will decide whether they are duplicates or not by comparing the representations.

In this month's column, I will provide some of the skeleton code functions which can help to implement this solution. I have deliberately not provided the complete code for the problem, as I would like readers to build their own solutions and become familiar with creating simple neural network models from scratch.

As discussed in last month's column, while there are multiple methods of creating a sentence representation, we can simply use a concatenation of word embeddings of the individual words in the question sentence to create a question embedding representation. We can either learn the word embeddings specific for the task of duplicate question-detection (provided our corpus is large and general enough), or we can use pre-trained word embeddings such as Word2vec.

In our example, we will use Word2vec embeddings and concatenate the embeddings of individual words to represent the question sentence. Assuming that we use Word2vec embeddings of dimension D, each input question can be represented by (NXD), where N is the number of words in the question and D is the embedding size. We need to feed in the input representation for each of the two questions we are comparing to the neural network model.

Before we look at the actual code for the neural network model, we need to decide which deep learning framework we would use to implement this model. There are a number of choices such as Theano, MxNet, Keras, CNTK, Torch, PyTorch, Caffe, Tensorflow, etc. A brief comparison of some of the popular deep learning frameworks is covered in the article at *https://deeplearning4j.org/compare-dl4j-tensorflow-pytorch*.

In selecting a deep learning framework for a project, one needs to consider both ease of programming, maintainability of code and long-term support for the framework. Some of the frameworks, such as Theano, are from academic groups; hence their support may be time-limited. For this project, we decided to go with TensorFlow, given its widespread adoption in the industry (it is sponsored by Google) and its ease of use. We will assume that our readers are familiar with TensorFlow (a quick introduction to it can be found at *https://www.tensorflow.org/get_started/get_started*).

Let us assume that we have a binary file which contains the word to embedding mapping for the Word2vec embeddings (pretrained word embeddings are available from either *https://code.google.com/archive/p/word2vec/* if you want to use the Word2vec model or *https://nlp.stanford.edu/projects/glove/* if you want to use the Glove vectors).

First, let's read our training corpus and build a vocabulary list that contains all the words in our training corpus. Next, let's build a map which maps each word to a valid Word2vec embedding. Shown below is the skeleton code for this:

```
def create_word_map(w2v_file, vocab_list):
data = np.load(w2v_file)
 glove_array = data['glove']
 for word in vocab_list:
     idx = vocab_list.index(word)
              w2v_wordmap[word] = glove_array[idx]
```